

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Департамент программной инженерии

ОТЧЁТ

по индивидуальному заданию №1
по дисциплине
«Архитектура компьютера и операционные системы»

Вариант 29. Группа лифтов

Выполнил: Буров Иван Васильевич
Группа: БПИ252

Москва, 2026

1 Условие задачи

В многоэтажном здании работает несколько лифтов под управлением общего диспетчера. Пассажиры появляются на разных этажах через случайные интервалы, каждый имеет номер и этаж назначения. Пассажир формирует вызов с этажом отправления и направлением; диспетчер назначает вызов лифту по выбранной стратегии. Лифт не принимает людей сверх вместимости: если прибывшая кабина заполнена, оставшиеся продолжают ожидание, и вызов считается выполненным только после формирования нового вызова для них. Лифт обслуживает попутные вызовы; направление меняется после выполнения заявок текущего направления.

Входные параметры: число этажей, лифтов и вместимость кабин; начальное положение лифтов; количество пассажиров и их этажи; диапазон времени появления; скорость движения и время дверей, посадки и высадки; стратегия назначения вызовов.

Инварианты: вместимость не превышает; пассажир входит и выходит только при открытых дверях на этаже остановки; лифт не движется с открытыми дверями; пассажир находится либо на этаже, либо в одном лифте; выходит только на заданном этаже; один вызов не назначается нескольким лифтам.

Завершение: после доставки всех пассажиров либо по истечении заданного периода; для оставшихся выводится состояние их заявок.

2 Отображение сущностей в программные конструкции

Задание требует последовательной имитации, поэтому участники модели отображаются не в процессы, а в структуры данных и функции, обрабатываемые по очереди на каждом такте. Время дискретно; все длительности заданы в тактах и являются входными параметрами.

Сущность	Конструкция	Содержание
Здание	Sim	всё состояние: кабины, пассажиры, реестр вызовов, время, счётчики
Лифт	Elevator	этаж, направление, состояние автомата, таймер фазы, признак открытых дверей, список едущих
Пассажир	Passenger	номер, откуда, куда, направление, статус, такты появления, посадки и доставки
Вызов	Call	этаж, направление, назначенный лифт, такт регистрации, признак активности
Диспетчер	dispatch	собственного состояния не имеет: раздаёт нераспределённые вызовы

Один такт (функция `sim_step`): появление очередного пассажира → назначение нераспределённых вызовов → обновление каждой кабины → проверка инвариантов → отрисовка и пауза.

Жизненный цикл вызова. Пассажир появляется и либо регистрирует вызов, либо присоединяется к существующему: на этаж и направление активен не более одного вызова. Диспетчер назначает его одной кабине и больше не переназначает. Кабина приезжает

и забирает *всех* ожидающих, едущих в её сторону, пока есть место, в том числе тех, чей вызов был назначен другой кабине — это и есть обслуживание попутных вызовов. Вызов гасится, а для не поместившихся немедленно регистрируется новый.

3 Структура программы и автомат кабины

Программа написана на C, собирается командой `make` с ключами `-std=c17 -D_POSIX_C_SOURCE=2008` предупреждений нет. Макрос `_POSIX_C_SOURCE` задан явно: при строгом `-std=c17` `glibc` не объявила бы `dprintf`.

Файл	Строк	Содержимое
<code>elevator.h</code>	102	типы, константы, объявления
<code>config.c</code>	250	умолчания, конфигурационный файл, аргументы, проверка значений
<code>sim.c</code>	396	пассажиры, вызовы, диспетчер, автомат кабины, инварианты
<code>output.c</code>	180	схема здания, лента событий, журнал, статистика
<code>main.c</code>	89	сигналы, главный цикл, причина завершения

Весь ввод-вывод идёт через файловые дескрипторы: `open`, `read`, `write`, `close`, `dprintf`. Потоки `FILE*` не используются нигде. Также применяются `sigaction` и `nanosleep`.

Автомат кабины состоит из шести состояний: `ST_IDLE`, `ST_MOVING` (длительность `travel_ticks`), `ST_OPENING` и `ST_CLOSING` (`door_ticks`), `ST_UNLOADING` и `ST_LOADING` (длительность пропорциональна числу входящих или выходящих). Такое устройство само по себе обеспечивает три инварианта: пассажир не может войти или выйти вне состояний посадки и высадки, а кабина не может ехать, не находясь в `ST_MOVING`.

Выбор направления. Функция `work_in_direction` отвечает, есть ли ещё причина ехать в текущую сторону: пассажир внутри, которому нужен этаж в этом направлении, либо назначенный кабине вызов с такого этажа. Пока причина есть, направление не меняется — это требование условия. Отдельного массива целевых этажей у кабины нет: вся информация уже содержится в списке едущих и реестре вызовов, а дублирующий массив пришлось бы синхронно поддерживать.

4 Диспетчер и проверка инвариантов

Диспетчер на каждом такте просматривает реестр и выбирает кабину для каждого нераспределённого вызова. Заполненные кабины в выборе не участвуют; если свободных нет, вызов рассматривается на следующем такте.

fcfs — вызовы обслуживаются в порядке поступления: берётся первая кабина без пассажиров и назначенных вызовов, а если все заняты — наименее загруженная.

nearest — берётся кабина, которая быстрее доберётся до этажа вызова. Оценка — расстояние в этажах, кабине, едущей в противоположную сторону, добавляется штраф в высоту здания.

Назначенный вызов никогда не переназначается. Это осознанное решение: именно оно делает невозможным нарушение шестого инварианта. Платой является неоптимальность — освободившаяся рядом кабина не перехватит вызов, назначенный дальней.

Инварианты проверяются во время работы; нарушение выводится строкой `INVARIANT BROKEN` и даёт код возврата 2. Четыре из них проверяются после каждого такта в `check_invariants`: вместимость, закрытые двери при движении, нахождение пассажира ровно в одной кабине, отсутствие двух активных вызовов на этаж и направление. Два оставшихся — вход и выход только при открытых дверях и выход только на своём этаже — проверяются в `do_boarding` и `do_unloading`, поскольку это единственные места, где пассажир меняет местоположение; по состоянию в конце такта проверить их уже невозможно.

5 Пользовательский интерфейс

Экран перерисовывается после каждого такта: сверху схема здания, снизу лента десяти последних событий.

```
tick 45    delivered 3/25    strategy nearest    in the building 4
```

f1	E1	E2	E3	hall
12	.	.	.	
11	.	.	.	
10	.	.	.	1v
9	.	.	[^0]	
8	.	.	.	
7	.	.	.	
6	.	.	.	
5	.	.	.	
4	[v1]	.	.	1^
3	.	.	.	
2	.	[^1]	.	
1	.	.	.	

```
[ 43] passenger 7 appears on floor 4 and wants floor 5 (up)
[ 43] call registered on floor 4, up
[ 43] dispatcher sends elevator 1 to the call on floor 4 (up)
[ 44] elevator 1 starts going down from floor 4
[ 45] elevator 3 reaches floor 9
```

Обозначения: `[^2]` — кабина с закрытыми дверями едет вверх, внутри двое; `[v2]` — вниз, `[-2]` — стоит; `(^2)` — двери открыты; точка — шахта на этаже пуста; столбец `hall` показывает, сколько человек ждёт и куда. Кабина в движении выделяется жёлтым, с открытыми дверями — зелёным. Скорость задаётся параметром `--delay-ms`, по умолчанию 200 мс. Флаг `--plain` отключает перерисовку и печатает события сплошной лентой: этот режим нужен для тестовых скриптов и сравнения журналов.

Журнал пишется параллельно экрану через отдельный дескриптор и содержит шапку с параметрами, все события и итоговую статистику.

6 Варианты запуска и завершения

Параметры берутся из трёх источников с возрастающим приоритетом: встроенные умолчания, конфигурационный файл, аргументы командной строки. Оба источника обраба-

тываются одной функцией `set_option`, поэтому набор ключей у них одинаков; в файле они пишутся через подчёркивание, в командной строке через дефис. Полный список выводится по `--help`. Все значения проверяются до начала моделирования: при недопустимом выводится диапазон и программа завершается с кодом 1, не начав работу.

```
./build/elevator                                # умолчания
./build/elevator --config data/rush-hour.conf    # сценарий из файла
./build/elevator --config data/default.conf --floors 20 --strategy fcfs
./build/elevator --plain --delay-ms 0 --seed 42 --log run.log
```

Предусмотрены три способа завершения, и все приводят к одинаковому финалу: выводится статистика и состояние незакрытых заявок.

1. **Все пассажиры доставлены** — естественное завершение.
2. **Исчерпан лимит тактов** `--max-ticks` — когда интересуется поведение системы за фиксированный период.
3. **Прерывание пользователем** — `SIGINT` и `SIGTERM` перехватываются через `sigaction`. Обработчик выставляет единственную переменную `volatile sig_atomic_t` и больше ничего не делает. Главный цикл видит флаг, выходит, восстанавливает курсор терминала, печатает статистику и закрывает журнал.

Коды возврата: 0 — норма, 1 — ошибка в параметрах, 2 — нарушен инвариант модели.

```
--- simulation finished: interrupted by the user ---
model time          41 ticks
delivered           2 of 25
average waiting     8.5 ticks
longest waiting     12 ticks
invariants broken   0
elevator 1          12 floors passed, 0 on board at the end

requests still open:
  passenger 3 waits on floor 7 for floor 3 since tick 14
  passenger 4 rides elevator 2 towards floor 6
  17 passenger(s) never appeared
```

7 Тестирование и результаты

В каталоге `data/` подготовлено пять сценариев, каждый является одновременно примером конфигурации и тестовым набором: `small` — 5 этажей и один лифт; `default` — 12 этажей, 3 лифта, вместимость 6; `rush-hour` — час пик, 16 этажей, 4 лифта, пассажир каждые 1–2 такта; `slow-doors` — то же здание, что базовое, но втрое медленнее двери и посадка; `crowded` — вместимость 1, почти каждый вызов приходится регистрировать заново.

Скрипт `./tests/run-tests.sh` пересобирает проект и выполняет 30 проверок: сборка без предупреждений; отказ на семи вариантах неверного ввода; прохождение всех пяти сценариев; проверка, что `crowded` действительно доводит до переполнения кабин; обе стратегии; краевые случаи (ноль пассажиров, два этажа при восьми лифтах, один лифт вместимостью один, обрыв по лимиту тактов); воспроизводимость по зерну в обе стороны; совпадение журнала с экраном; обработка `SIGINT`; 60 прогонов со случайными конфигурациями здания. Скрипт возвращает 0, только если прошли все.

Генератор случайных чисел реализован в программе собственный, а не взят из библиотеки: **rand** не обязана давать одинаковую последовательность на разных системах, а для сравнения журналов между прогонами это необходимо.

Сценарий	Тактов	Доставлено	Ожидание	Макс.	Поездка	Отказов
small	65	8/8	15,0	27	8,6	0
default	191	25/25	13,4	36	14,8	0
rush-hour	151	60/60	17,2	59	16,8	0
slow-doors	227	25/25	43,1	100	31,4	0
crowded	223	20/20	70,2	206	12,8	21

Времена указаны в тактах. Результаты согласуются со здравым смыслом: в **slow-doors** здание и поток те же, что в базовом сценарии, но все длительности увеличены втрое — среднее ожидание выросло с 13,4 до 43,1 такта. В **crowded** вместимость равна одному человеку, ожидание подскакивает до 70 тактов при 21 отказе в посадке, тогда как сама поездка остаётся самой короткой: здание низкое.

Сравнение стратегий на одном потоке пассажиров (`--seed 42`, базовое здание): **fcfs** — 145 тактов, среднее ожидание 14,3; **nearest** — 135 тактов, среднее ожидание 15,2. Стратегия **nearest** завершает обслуживание раньше, но среднее ожидание у неё чуть выше: отправляя к вызову ближайшую кабину, она группирует кабины в одной части здания, и дальние вызовы ждут дольше. На одном прогоне делать выводы некорректно, для содержательного сравнения нужно усреднение по многим зёрнам.

Во всех 60 прогонах со случайными конфигурациями (2–26 этажей, 1–8 лифтов, вместимость 1–6) все пассажиры доставлены, счётчик нарушений инвариантов остался нулевым.

8 Выводы

Разработано консольное приложение, моделирующее работу группы лифтов под управлением общего диспетчера: один процесс, один поток, дискретное модельное время. Реализованы все требования условия — вызовы с этажом и направлением, две стратегии диспетчеризации, ограничение вместимости с перерегистрацией вызова для не поместившихся, обслуживание попутных вызовов, смена направления только после отработки заявок текущего направления. Все шесть инвариантов проверяются во время работы. Предусмотрены три способа завершения, включая прерывание пользователем, и во всех случаях выводится состояние незакрытых заявок. Корректность подтверждена набором из 30 автоматических проверок.

9 Использовани ИИ

Использовал нейросеть Claude (Anthropic) для помощи в организации архитектуры проекта, добавления комментариев, составления тестов и отчета.